Exception Handling in Python

---

1. Introduction to Exception Handling

Every program may face unexpected situations while running — such as invalid input, missing files, or mathematical errors like division by zero.
When such problems occur, Python raises an **exception**. If not handled properly, the program immediately stops execution and displays an error message.

Exception Handling allows programmers to handle these errors gracefully, so the program continues to run or terminates cleanly without crashing.

---

2. Difference Between Errors and Exceptions

- **Error:** Problems that occur at compile-time (before the program runs). For example, syntax errors like missing a colon or parentheses.
- **Exception:** Problems that occur during program execution (runtime). For example, dividing by zero, or converting an invalid string to an integer.

Example:

```
# Syntax Error (compile-time)

print("Hello"


# Exception (runtime)

print(10 / 0)
```

In the first case, Python won't even execute the code because of a missing parenthesis.
In the second case, the program runs but stops when it encounters division by zero.

---

3. Why Exception Handling is Needed

If exceptions are not handled:

- The entire program stops abruptly.
- The user gets a technical and confusing error message.
- Any remaining code does not execute.

When handled properly:

- The error can be caught and managed.
- The program can continue executing remaining statements.
- Users see clear and friendly messages.

Without Handling:

```
a = int(input("Enter a number: "))
b = int(input("Enter another number: "))
print(a / b)     # Program will crash if b = 0
```

With Handling:

```
try:
    a = int(input("Enter a number: "))
    b = int(input("Enter another number: "))
    print(a / b)
except ZeroDivisionError:
    print("Cannot divide by zero. Please enter a valid number.")
```

---

4. The Try and Except Block

The basic structure of exception handling in Python uses two keywords: try and except.

Syntax:

```
try:
    # Code that may cause an exception
except:
    # Code to handle the exception
```

When Python executes the code inside the try block, if an error occurs, control immediately jumps to the except block.

Example:

```
try:
    num = int(input("Enter a number: "))
    print(10 / num)
except:
    print("Something went wrong!")
```

This structure prevents the program from terminating abruptly and provides a controlled way to respond to errors.

---

5. Handling Specific Exceptions

Python allows handling of specific exception types. This is considered best practice because it gives more precise control and helps in debugging.

Example:

```
try:
    num1 = int(input("Enter numerator: "))
    num2 = int(input("Enter denominator: "))
    result = num1 / num2
    print("Result =", result)
except ZeroDivisionError:
    print("Error: Division by zero is not allowed.")
except ValueError:
    print("Error: Please enter numeric values only.")
```

Explanation

- ZeroDivisionError: Raised when dividing by zero.
- ValueError: Raised when the input cannot be converted to an integer.

If a user enters invalid input like "abc", the ValueError block executes.
If a user enters 10 and 0, the ZeroDivisionError block executes.

---

6. Using Multiple Except Blocks

Sometimes, different parts of the same try block can raise different types of errors. You can use multiple except blocks to handle them separately.

Example:

```
try:
    a = [1, 2, 3]
    index = int(input("Enter an index: "))
    print(a[index])
    print(10 / index)
except IndexError:
    print("Error: Index out of range.")
except ZeroDivisionError:
    print("Error: Division by zero.")
except ValueError:
```

```
    print("Error: Please enter a valid integer.")
```

This structure makes the program robust and easy to debug since each exception is handled specifically.

---

7. The Else Block

The else block runs only if the try block executes without raising any exception. It helps to write code that should run only when no error occurs.

Example:

```
try:
    num = int(input("Enter a number: "))
    print("Square of the number is", num * num)
except ValueError:
    print("Invalid input. Please enter a number.")
else:
    print("No error occurred.")
```

If the user enters a valid number, both the try and else blocks run. If an error occurs, the except block runs, and the else block is skipped.

---

8. The Finally Block

The finally block is always executed — whether an exception occurs or not. It is generally used to release resources, close files, or clean up data.

Example:

```
try:
    f = open("example.txt", "r")
    print(f.read())
except FileNotFoundError:
    print("File not found.")
finally:
    print("File handling process complete.")
```

Here, even if the file is missing, the finally block still executes, ensuring that the cleanup code always runs.

---

9. Nested Try-Except Blocks

A try block can contain another try block inside it. This is called **nested exception handling**, and it's useful when a smaller part of a large block might raise a different error.

Example:

```
try:
    a = int(input("Enter a number: "))
    try:
        b = int(input("Enter another number: "))
        print("Result:", a / b)
    except ZeroDivisionError:
        print("Inner error: Cannot divide by zero.")
except ValueError:
    print("Outer error: Invalid input. Please enter integers only.")
```

This helps handle different types of exceptions at different levels of code execution.

---

10. Raising Exceptions Manually

Python allows you to raise exceptions manually using the raise keyword. This is useful for enforcing rules or conditions in your program.

Example:

```
age = int(input("Enter your age: "))
if age < 18:
    raise ValueError("Age must be at least 18.")
else:
    print("You are eligible to vote.")
```

Here, the program intentionally raises an error if the age condition fails.

---

11. Common Built-in Exceptions in Python

Exception Name	**Description**
ZeroDivisionError	Division or modulo by zero
ValueError	Invalid literal or conversion
TypeError	Invalid operation between data types
IndexError	Invalid list or tuple index

KeyError	Missing key in dictionary
FileNotFoundError	File not found during open operation
AttributeError	Accessing invalid attribute of an object
NameError	Variable not defined
IOError	Input/output operation fails
ImportError	Module not found

---

12. Exception Handling in File Operations

File operations are one of the most common sources of runtime errors — for example, when a file is missing or cannot be opened.

Example:

```
try:
    f = open("students.txt", "r")
    data = f.read()
    print(data)
except FileNotFoundError:
    print("The specified file was not found.")
except PermissionError:
    print("You do not have permission to access this file.")
finally:
    print("File operation completed.")
```

---

13. Combining Else and Finally

It is possible to use both else and finally together for more structured handling.

Example:

```
try:
    n = int(input("Enter a number: "))
    result = 100 / n
except ZeroDivisionError:
    print("Cannot divide by zero.")
except ValueError:
```

```
    print("Invalid input.")
else:
    print("Division successful. Result =", result)
finally:
    print("Program execution finished.")
```

Here, else runs only if no exception occurs, and finally always executes at the end.

---

14. Summary of Exception Handling Blocks

Block	Description
try	Contains code that may cause an exception
except	Handles the exception that occurs in try block
else	Executes only when no exception occurs
finally	Executes regardless of whether exception occurs
raise	Used to manually raise an exception
